UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ ACCREDITED UNIVERSITY

Worksheet-7

Name: TANUJ JOSHI
UID: 25MCA20081
Branch: MCA General
Section/Group: 25MCA_KAR-1
Semester: II
Date of Performance: 31-03-26
Subject Name: Technical Training
Subject Code: 25CAP-652

Aim of the Session:

The aim of this practical session is to understand and implement different types of SQL joins to retrieve related data from multiple tables. Joins are essential for combining data stored across different tables and are widely used in relational database systems.

Objective of the Session:

The objectives of this practical session are:

- To understand different types of joins (INNER, LEFT, RIGHT, SELF, CROSS).
- To retrieve meaningful data from multiple related tables.
- To identify unmatched and matched records using joins.
- To analyze relationships between tables using SQL queries.
- To gain practical exposure to real-world database querying techniques.

Practical / Experiment Steps:

Step 1: Creating tables for Students, Courses, and Enrollments.
Step 2: Using INNER JOIN to fetch students with their enrolled courses.
Step 3: Using LEFT JOIN to find students not enrolled in any course.
Step 4: Using RIGHT JOIN to display all courses with or without students.
Step 5: Using SELF JOIN / Multiple JOIN to show student-department relationships.
Step 6: Using CROSS JOIN to display all possible student-course combinations.

Procedure of the Practical:

1. Start the system and log in to the computer.
2. Open the PostgreSQL client tool (psql / pgAdmin).
3. Create Tables.

```
CREATE TABLE Departments (
   DeptID SERIAL PRIMARY KEY,
   DeptName VARCHAR(50)
);

CREATE TABLE Students (
   StudentID SERIAL PRIMARY KEY,
   StudentName VARCHAR(50),
   DeptID INT REFERENCES Departments(DeptID)
);

CREATE TABLE Courses (
   CourseID SERIAL PRIMARY KEY,
   CourseName VARCHAR(50)
);

CREATE TABLE Enrollments (
   EnrollID SERIAL PRIMARY KEY,
   StudentID INT REFERENCES Students(StudentID),
   CourseID INT REFERENCES Courses(CourseID)
);
```

4. Insert Sample Data

```
INSERT INTO Departments (DeptName) VALUES
('Computer Science'),
('Management'),
('Commerce');

INSERT INTO Students (StudentName, DeptID) VALUES
('Amit', 1),
('Riya', 2),
('Kunal', 1),
('Sneha', 3),
('Rahul', 2);

INSERT INTO Courses (CourseName) VALUES
('DBMS'),
('Java'),
('Accounting'),
('Marketing');
```

```
INSERT INTO Enrollments (StudentID, CourseID) VALUES
(1,1),
(1,2),
(2,4),
(3,1),
(4,3);
```

5. INNER JOIN (Students with Enrolled Courses)

```
SELECT s.StudentName, c.CourseName
FROM Students s
INNER JOIN Enrollments e ON s.StudentID = e.StudentID
INNER JOIN Courses c ON e.CourseID = c.CourseID;
```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	studentname character varying (50)	coursename character varying (50)
1	Amit	DBMS
2	Amit	Java
3	Riya	Marketing
4	Kunal	DBMS
5	Sneha	Accounting

6. LEFT JOIN (Students not Enrolled in Any Course)

```
SELECT s.StudentName
FROM Students s
LEFT JOIN Enrollments e ON s.StudentID = e.StudentID
WHERE e.StudentID IS NULL;
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	studentname character varying (50)
1	Rahul

7. RIGHT JOIN (All Courses with or without Students)

```
SELECT c.CourseName, s.StudentName
FROM Enrollments e
RIGHT JOIN Courses c ON e.CourseID = c.CourseID
LEFT JOIN Students s ON e.StudentID = s.StudentID;
```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	coursename character varying (50)	studentname character varying (50)
1	DBMS	Amit
2	Java	Amit
3	Marketing	Riya
4	DBMS	Kunal
5	Accounting	Sneha

8. SELF JOIN / MULTIPLE JOIN (Students with Department Info)

```
SELECT s.StudentName, d.DeptName
FROM Students s
JOIN Departments d ON s.DeptID = d.DeptID;
```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	studentname character varying (50)	deptname character varying (50)
1	Amit	Computer Science
2	Riya	Management
3	Kunal	Computer Science
4	Sneha	Commerce
5	Rahul	Management

9. CROSS JOIN (All Possible Student-Course Combinations)

```
SELECT s.StudentName, c.CourseName
FROM Students s
CROSS JOIN Courses c;
```

Data Output Messages Notifications

Showing rows: 1 to 20 Page No: 1 of 1

	studentname character varying (50)	coursename character varying (50)
1	Amit	DBMS
2	Riya	DBMS
3	Kunal	DBMS
4	Sneha	DBMS
5	Rahul	DBMS
6	Amit	Java

10. Verify the output after execution.
11. Note down the results obtained.
12. Save the work and take screenshots for record.

I/O Analysis:

Input Provided:

- SQL JOIN queries
- Student, Course, Department, and Enrollment tables

Output Generated:

- Student-course mapping using INNER JOIN
- Students without courses using LEFT JOIN
- All courses using RIGHT JOIN
- Student-department data using JOIN
- Complete combinations using CROSS JOIN

Learning Outcomes:

- Understanding different types of SQL joins.
- Ability to combine multiple tables effectively.
- Knowledge of handling missing and matching data.
- Practical exposure to relational database queries.
- Improved skills in writing complex SQL queries.